Mise en place d'un écosystème d'IA — 420-A57-BB

TRAVAIL PRATIQUE

Module 5 : Contrôler et entretenir l'infrastructure IA

Professeur : Dr Komi SODOKE

Environnement	Google Colab (GPU T4 activé)
Objectif	Apprendre à surveiller un système IA et détecter les problèmes

Partie 1 — Surveiller le GPU en temps réel

Exercice 1.1 — Découvrir nvidia-smi sur Colab

Activez le GPU (Exécution → Modifier le type → GPU T4), puis exécutez :

Cellule Colab

```
# Snapshot du GPU
!nvidia-smi

# Version structurée
!nvidia-smi --query-
gpu=name,temperature.gpu,utilization.gpu,memory.used,memory.total,power.draw \
  --format=csv,noheader
```

Question 1 : Quel GPU avez-vous ? Quelle température affiche-t-il au repos ? Combien de VRAM est utilisée avant tout entraînement ?

Exercice 1.2 — Monitoring pendant un entraînement

Lancez un entraînement et observez l'impact sur le GPU :

Cellule Colab

```
import torch, torch.nn as nn, time, threading

# Fonction de monitoring en arrière-plan
def monitor_gpu(interval=5, duration=60):
    print('Timestamp | Temp(°C) | GPU(%) | VRAM(Mo)')
    print('-' * 50)
    for _ in range(duration // interval):
        import subprocess
        r = subprocess.run(['nvidia-smi','--query-
gpu=temperature.gpu,utilization.gpu,memory.used',
                            '--format=csv,noheader,nounits'], capture_output=True,
text=True)
        vals = r.stdout.strip()
        print(f'{time.strftime("%H:%M:%S")} | {vals}')
        time.sleep(interval)

# Lancer le monitoring
t = threading.Thread(target=monitor_gpu, args=(5, 120))
t.start()

# Simuler un entraînement GPU intensif
model = nn.Sequential(nn.Linear(1024, 2048), nn.ReLU(), nn.Linear(2048,
1024)).cuda()
opt = torch.optim.Adam(model.parameters())
for i in range(2000):
    x = torch.randn(256, 1024, device='cuda')
    loss = model(x).sum()
```

```
    loss.backward()
    opt.step()
    opt.zero_grad()

print('Entraînement terminé !')
t.join()
```

Question 2 : Comment la température, l'utilisation GPU et la VRAM ont-elles évolué pendant l'entraînement ? Quand le GPU est-il le plus sollicité ?

Partie 2 — Métriques système complètes

Exercice 2.1 — Script de diagnostic complet

Créez un script qui collecte toutes les métriques système en un coup :

Cellule Colab

```
import os, platform, shutil, subprocess

def diagnostic_complet():
    print('=' * 60)
    print('DIAGNOSTIC SYSTÈME IA')
    print('=' * 60)

    # OS
    print(f'\nOS : {platform.platform()}')
    print(f'Python : {platform.python_version()}')

    # CPU
    print(f'\nCPU : {os.cpu_count()} cœurs')
    with open('/proc/loadavg') as f:
        load = f.read().split()[:3]
    print(f'Load Average (1/5/15 min) : {" / ".join(load)}')

    # RAM
    import psutil
    ram = psutil.virtual_memory()
    print(f'\nRAM : {ram.used/1e9:.1f} / {ram.total/1e9:.1f} Go
({ram.percent}%)')
    swap = psutil.swap_memory()
    print(f'Swap : {swap.used/1e9:.1f} / {swap.total/1e9:.1f} Go')

    # Disque
    disk = shutil.disk_usage('/')
    print(f'\nDisque : {disk.used/1e9:.1f} / {disk.total/1e9:.1f} Go
({100*disk.used/disk.total:.0f}%)')

    # GPU
    try:
        r = subprocess.run(['nvidia-smi','--query-
gpu=name,temperature.gpu,utilization.gpu,memory.used,memory.total',
                           '--format=csv,noheader'], capture_output=True,
text=True)
        print(f'\nGPU : {r.stdout.strip()}')
    except: print('\nGPU : Non disponible')

    # Évaluation
    print('\n' + '=' * 60)
    alerts = []
    if ram.percent > 90: alerts.append('⚠ RAM > 90%')
    if 100*disk.used/disk.total > 90: alerts.append('⚠ Disque > 90%')
```

```
    if float(load[0]) > os.cpu_count(): alerts.append('⚠ CPU surchargé')
    if alerts:
        for a in alerts: print(a)
    else:
        print(' Tous les indicateurs sont normaux')

!pip install psutil -q
diagnostic_complet()
```

Question 3 : Quels sont les résultats de votre diagnostic ? Y a-t-il des alertes ? Comment ce script pourrait-il être amélioré (indice : seuils personnalisables, envoi d'email) ?

Exercice 2.2 — Détecter les goulots d'étranglement

Simulons un goulot d'étranglement I/O pour comprendre l'impact :

Cellule Colab

```
import time, torch
from torch.utils.data import DataLoader, TensorDataset

# Simuler un dataset
X = torch.randn(10000, 784)
y = torch.randint(0, 10, (10000,))
dataset = TensorDataset(X, y)

# Test 1 : DataLoader lent (1 worker)
loader_slow = DataLoader(dataset, batch_size=64, num_workers=0)
start = time.time()
for batch_x, batch_y in loader_slow: pass
t_slow = time.time() - start

# Test 2 : DataLoader rapide (multi-workers)
loader_fast = DataLoader(dataset, batch_size=64, num_workers=2)
start = time.time()
for batch_x, batch_y in loader_fast: pass
t_fast = time.time() - start

print(f'1 worker  : {t_slow:.3f}s')
print(f'2 workers : {t_fast:.3f}s')
print(f'Gain : {t_slow/t_fast:.1f}x plus rapide')
```

Question 4 : Quel est le gain de vitesse avec plus de workers ? Pourquoi le nombre de workers du DataLoader impacte-t-il la vitesse d'entraînement ? Quel est le lien avec le monitoring CPU ?

Partie 3 — Checkpoints et récupération

Exercice 3.1 — Sauvegarder et reprendre un entraînement

Apprenez à sauvegarder des checkpoints pour survivre aux pannes :

Cellule Colab

```
import torch, torch.nn as nn

# Modèle simple
model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Simuler 5 epochs d'entraînement
for epoch in range(5):
    # (simulation de l'entraînement)
    fake_loss = 1.0 / (epoch + 1)
    print(f'Epoch {epoch+1}/5 — Loss: {fake_loss:.4f}')

    # SAUVEGARDER LE CHECKPOINT
    checkpoint = {
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'loss': fake_loss,
    }
    torch.save(checkpoint, f'checkpoint_epoch_{epoch+1}.pt')
    print(f'  → Checkpoint sauvegardé')

print('\nEntraînement terminé !')

# Vérifier les fichiers
import os
for f in sorted(os.listdir('.')):
    if f.endswith('.pt'):
        size = os.path.getsize(f) / 1024
        print(f'{f} : {size:.0f} Ko')
```

Exercice 3.2 — Reprendre depuis un checkpoint

Cellule Colab

```
# Simuler un crash : on crée un NOUVEAU modèle (vierge)
new_model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
new_optimizer = torch.optim.Adam(new_model.parameters())

# REPRENDRE depuis le checkpoint epoch 3
checkpoint = torch.load('checkpoint_epoch_3.pt')
new_model.load_state_dict(checkpoint['model_state_dict'])
new_optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
start_epoch = checkpoint['epoch'] + 1
last_loss = checkpoint['loss']
```

```
print(f'Reprise depuis epoch {start_epoch}')
print(f'Dernière loss : {last_loss:.4f}')
print('Le modèle est restauré — on peut continuer l\'entraînement !')
```

Question 5 : Que contient un checkpoint ? Pourquoi sauvegarder aussi l'optimizer (pas seulement le modèle) ? À quelle fréquence recommandez-vous de sauvegarder les checkpoints pour un entraînement de 100 epochs ?

Partie 4 — Questions de synthèse

Q1 : Qu'est-ce que le monitoring et pourquoi est-il indispensable en IA ? Citez 3 outils de monitoring.

Q2 : Votre GPU affiche 87°C pendant un entraînement. Listez 3 causes possibles et 3 actions correctives.

Q3 : Expliquez la différence entre Prometheus et Grafana. Quel est le rôle de chacun ?

Q4 : Qu'est-ce qu'un watchdog ? Donnez un exemple avec Docker.

Q5 : Votre entraînement crashe à l'epoch 47/50. Vous n'avez pas de checkpoint. Que se passe-t-il ? Comment éviter cette situation ?

Q6 : Proposez un calendrier de maintenance (quotidien, hebdomadaire, mensuel) pour un serveur IA.